ZEN MOBILE BROWSER

AI Build Prompt — Usable Proof of Concept

React Native · Firefox Sync · Multi-Workspace · Gesture Navigation

---

This document is a complete engineering prompt intended to be given to an AI coding agent (such as Claude, Cursor, or GPT-4o in agent mode) or used directly by a developer to build a functional, installable proof-of-concept of the Zen Mobile Browser. Every section below is a direct instruction. The goal is a working React Native app that demonstrates all core interactions, compiles on iOS and Android, and leaves the architecture clean enough to iterate on.

1. Project Overview & Goal

Build a mobile browser application in React Native that brings the Zen Browser philosophy — pioneered on desktop as a Firefox fork — to iOS and Android. The browser should feel native, fast, and privacy-respecting. For this proof of concept, the actual web rendering is delegated to a WebView component; the primary deliverable is the complete shell, navigation paradigm, workspace system, and sync scaffolding.

The POC must be installable and runnable via Expo Go or a local dev build. It does not need a deployed backend — the Firefox Sync integration should be wired to real FxA OAuth endpoints but can operate in a degraded/mock state if authentication fails during testing.

Parameter	Value
Framework	React Native 0.73+ with Expo (managed or bare workflow)
Target platforms	iOS 16+ and Android 12+
Language	TypeScript — strict mode enabled throughout
State management	Zustand for local state; no Redux
Navigation	React Navigation v6 with bottom-tab and stack navigators
Persistence	MMKV (react-native-mmkv) for sync-state; AsyncStorage for everything else
Gestures	react-native-gesture-handler + react-native-reanimated v3
WebView	react-native-webview (latest stable)

2. Architecture Overview

The application is divided into four clearly separated layers. Understanding this separation is critical before writing any code, because it determines where each feature lives and how the codebase stays navigable as the POC grows.

Layer 1 — Engine (WebView)

The WebView layer is a thin wrapper around react-native-webview. Each browser tab is its own WebView instance, but only the active tab is rendered; inactive tabs are suspended and their state (URL, scroll position, title, favicon) is persisted in the tab store. On Android, GeckoView can be substituted later via a native module, but the POC uses the platform default (Chromium on Android, WebKit on iOS) to remain buildable without native code.

Layer 2 — Shell (React Native UI)

The shell is everything the user sees that is not a webpage: the URL bar, the tab tray, workspace chips, the menu sheet, and gesture overlays. All shell components live in src/components/shell/. They are gesture-driven and animated with Reanimated shared values. The shell communicates with the engine layer only through the tab store.

Layer 3 — Store (Zustand)

A single Zustand store (src/store/browserStore.ts) holds the entire application state: the list of workspaces, the tabs within each workspace, the active tab per workspace, and the active workspace itself. The store is the single source of truth. No component should derive tab state from props — always read from the store. The store is persisted to MMKV so that a cold restart restores the previous session.

Layer 4 — Sync (Firefox Account)

The sync layer handles authentication against Mozilla's Firefox Accounts service and the storage sync API. For the POC, this layer is implemented as a service module (src/services/fxaService.ts) that exposes four functions: signIn, signOut, push, and pull. The push/pull operations serialize the workspace and tab store to the FxA storage sync collection. If the user is not signed in, the app continues to work fully in offline mode.

---

3. File & Folder Structure

Create the project with the following structure. Do not deviate from it — keeping this layout consistent makes the prompt reproducible across different AI agents.

```
zen-mobile/
├── app.json
├── App.tsx                    ← root navigator
├── src/
│   ├── components/
│   │   ├── shell/
```

```
│   │   │   ├── UrlBar.tsx
│   │   │   ├── TabTray.tsx
│   │   │   ├── TabCard.tsx
│   │   │   ├── WorkspaceChips.tsx
│   │   │   └── MenuSheet.tsx
│   │   └── browser/
│   │       ├── BrowserView.tsx
│   │       └── WebViewWrapper.tsx
│   ├── screens/
│   │   ├── BrowserScreen.tsx  ← main screen
│   │   ├── NewTabScreen.tsx
│   │   └── SettingsScreen.tsx
│   ├── store/
│   │   ├── browserStore.ts
│   │   └── types.ts
│   ├── services/
│   │   ├── fxaService.ts
│   │   └── syncService.ts
│   ├── hooks/
│   │   ├── useGestures.ts
│   │   └── useWebView.ts
│   └── theme/
│       └── index.ts           ← all colors, spacing, radius
```

---

4. Store Schema (src/store/types.ts)

Define these TypeScript interfaces first, before writing any component. Every other file in the project imports from this module.

```
interface Tab {
  id: string;          // uuid v4
  workspaceId: string;
  url: string;
  title: string;
  favicon: string | null;
  isPinned: boolean;
  isLoading: boolean;
  canGoBack: boolean;
  canGoForward: boolean;
  scrollY: number;
  createdAt: number;   // Date.now()
}
```

```
interface Workspace {
  id: string;
  label: string;
  emoji: string;
  color: string;      // hex, used for accent on workspace chip
  tabIds: string[];   // ordered list of tab ids
  activeTabId: string | null;
}

interface BrowserState {
  workspaces: Record<string, Workspace>;
  workspaceOrder: string[];   // display order
  tabs: Record<string, Tab>;  // flat map, keyed by tab.id
  activeWorkspaceId: string;
  isTrayOpen: boolean;
  isMenuOpen: boolean;
  syncUser: { email: string; uid: string } | null;
  lastSyncedAt: number | null;
}
```

---

5. Feature Specifications

5.1 Gesture Navigation

Gestures are the core interaction model. Implement all of the following using react-native-gesture-handler's PanGestureHandler combined with Reanimated shared values. Do not use the Animated API from React Native core — use Reanimated exclusively for performance.

- Swipe up on the browser content area (minimum 60px vertical travel, velocity > 200) opens the tab tray with a spring animation.
- Swipe down on the tab tray closes it.
- Swipe left on the browser content area navigates to the next tab in the current workspace (wraps around).
- Swipe right on the browser content area navigates to the previous tab.
- The URL bar should animate downward by 8px with a subtle scale (0.97) when the tray opens, reinforcing the layering.
- Add haptic feedback (expo-haptics) on workspace switch and on tab close.

  IMPORTANT: *Gestures must be carefully nested. The WebView consumes all touch events by default. Wrap the WebView in a View with pointerEvents='box-none' and attach gestures to that outer View, not the WebView itself. Use simultaneousHandlers where the WebView scroll and the swipe-up gesture need to coexist.*

5.2 Tab Tray

The tab tray slides up from behind the URL bar and displays all tabs in the active workspace as horizontally scrollable cards. Each card is 120px wide and 150px tall.

- Each card shows the favicon (or a letter avatar if null), the page title (truncated to 2 lines), and the domain.
- The active tab card has a 2px accent border in the workspace color.
- Pinned tabs display a pin icon in the top-right corner.
- A close button appears on long-press of a card (not on tap, to prevent accidental closes).
- A 'New Tab' card at the end of the list opens a new tab in the current workspace.
- The tray header contains the WorkspaceChips row (see 5.3) and a tab count label.

5.3 Workspace System

Workspaces are named groups of tabs, each with an emoji and an accent color. Think of them as tab containers, similar to Firefox's container tabs but with a stronger visual identity.

- Three default workspaces are created on first launch: Personal (🏠 blue), Work (💼 indigo), Research (🔬 green).
- The user can add a new workspace from the menu sheet (emoji picker + label input + color picker with 8 preset colors).
- The active workspace is shown in the URL bar as a small emoji badge on the right side of the URL pill.
- Tapping the emoji badge opens the tab tray directly to the workspace chips row.
- Switching workspace instantly updates the URL bar domain and triggers a brief workspace name toast.
- Each workspace maintains its own active tab state independently.

5.4 URL Bar

The URL bar sits at the bottom of the screen and is always visible. It is the anchor of the entire UI.

- The bar contains, left to right: a new-tab '+' button, a URL pill, a tab-count button, and a menu '···' button.
- The URL pill displays the lock icon, the bare domain of the active URL (not the full URL), and the workspace emoji badge.
- Tapping the URL pill opens a full-screen address bar overlay with a TextInput pre-filled with the current full URL, a list of history suggestions filtered in real-time, and a list of top bookmarks.
- The address bar overlay can be dismissed by tapping outside it or pressing the device back button.
- When a new URL is submitted, validate it: if it contains a dot and no spaces, treat it as a URL (prepend https:// if missing); otherwise, treat it as a search query and open

https://search.brave.com/search?q={query} (use Brave Search as the privacy-respecting default).

- The tab-count button shows the count in a small badge and opens the tray on tap.

5.5 Menu Sheet

The menu sheet slides up from the bottom when the '⋯' button is tapped. It is a bottom sheet with a drag handle.

- Top section: Firefox Account card. If signed in, shows the avatar initial, email address, and a green 'Synced' badge with the last sync time. If not signed in, shows a 'Sign in to Firefox' CTA button.
- Navigation row: Back and Forward buttons (enabled/disabled based on the active WebView's canGoBack/canGoForward state).
- Actions grid (2 columns): Bookmark current tab, Share URL (uses native share sheet), Reload, Find in page, Desktop site toggle, Settings.
- Bottom: 'New Workspace' button.

5.6 Firefox Sync

Implementing Firefox Sync correctly is the most complex part of the project. The following describes the minimal viable integration for the POC.

Authentication flow

- Use expo-web-browser to open the Firefox Accounts OAuth endpoint in an in-app browser session.
- The OAuth redirect URI should be zen-mobile://fxa-oauth. Register this scheme in app.json.
- On successful auth, you receive an authorization code. Exchange it for session tokens via the FxA token endpoint (https://oauth.accounts.firefox.com/v1/token).
- Store the access token, refresh token, and UID securely using expo-secure-store. Never store credentials in MMKV or AsyncStorage.

Data sync

- The sync payload is the full BrowserState (workspaces + tabs) serialized to JSON, then encrypted client-side using AES-256-GCM before upload.
- Use the Firefox Sync Storage API (https://sync-1-us-west4-g.sync.services.mozilla.com/) to push and pull a single record under the collection 'zen-mobile-state'.
- Sync is triggered: on app foreground (if last sync was > 5 minutes ago), on workspace creation, and manually from the menu sheet.
- Conflict resolution: last-write-wins based on the record's modified timestamp.

  NOTE: *The FxA/Sync API requires HAWK authentication headers. Use the hawk npm package to generate these. The sync service endpoints require the x-keyID and x-weave-records headers — refer to the Firefox Sync 1.5 API documentation for exact header requirements.*

5.7 Bookmarks

Bookmarks are stored locally in the Zustand store (persisted to MMKV) and synced as part of the state payload.

- A bookmark has: id, url, title, favicon, folderId (nullable), createdAt.
- A default 'Unsorted' folder exists. Users can create additional folders from the bookmarks screen.
- The 'Bookmark current tab' action in the menu sheet adds the active tab to 'Unsorted'. A visual confirmation toast appears.
- The bookmarks screen is accessible from Settings. It shows a list grouped by folder.

5.8 History

Browser history is stored as a flat array in the Zustand store, capped at 500 entries, oldest entries pruned first.

- Each history entry has: id, url, title, visitedAt (timestamp).
- History is used to power the autocomplete suggestions in the address bar overlay. Filter entries whose title or URL contains the typed string (case-insensitive, debounced 150ms).
- The history screen is accessible from Settings.

---

6. Settings Screen

Keep the settings screen simple. It exists primarily to expose the sync and privacy controls. Implement the following settings grouped into sections:

Account

- Sign in / Sign out of Firefox Account.
- Sync now button.
- Last synced timestamp.

Privacy

- Default search engine selector (Brave Search, DuckDuckGo, Google, Bing).
- Block trackers toggle (inject a basic content blocker script into the WebView via injectedJavaScript — block the top 20 known tracker domains by refusing to load them with a navigation state change listener).
- Clear browsing data: history, cache (reloads all WebViews), cookies (not possible without a native module on iOS; show an informational alert explaining the limitation).

Appearance

- Dark / Light / System theme toggle. Store the preference and apply it by swapping a theme context value at the app root.

About

- App version from app.json, a link to the project GitHub, and an 'Open Source Licenses' screen (use expo-constants).

---

7. Theme System (src/theme/index.ts)

Define the entire visual language as a typed theme object. Never hardcode colors or spacing values in component files. Import from the theme exclusively.

```
export const darkTheme = {
  bg:       '#0d0f14',
  surface:  '#141720',
  surface2: '#1c2030',
  border:   'rgba(255,255,255,0.07)',
  accent:   '#6c9fff',
  accent2:  '#a78bfa',
  text:     '#e8eaf2',
  text2:    '#7a809a',
  text3:    '#454b66',
  danger:   '#ff6b6b',
  radius:   { sm: 8, md: 14, lg: 20, pill: 100 },
  spacing:  { xs: 4, sm: 8, md: 14, lg: 20, xl: 32 },
};
```

The light theme should be a separate export (lightTheme) with inverted background and surface values. Wrap the app in a ThemeContext provider and expose a useTheme() hook.

---

8. Dependencies — Exact Package List

Install the following packages. Use the exact versions listed to avoid peer dependency conflicts. Run the install command in one shot.

```
npx create-expo-app zen-mobile --template blank-typescript
cd zen-mobile
npx expo install \
  react-native-webview \
  react-native-gesture-handler \
  react-native-reanimated \
```

```
  react-native-safe-area-context \
  react-native-screens \
  @react-navigation/native \
  @react-navigation/bottom-tabs \
  @react-navigation/stack \
  zustand \
  react-native-mmkv \
  expo-haptics \
  expo-secure-store \
  expo-web-browser \
  expo-linking \
  expo-constants \
  react-native-safe-area-context \
  uuid

npm install hawk
```

NOTE: *react-native-mmkv requires a bare workflow (not Expo Go). If you want to test in Expo Go first, replace MMKV with @react-native-async-storage/async-storage and swap it out later. The store interface should not change — only the persistence adapter.*

---

9. Key Implementation Details

9.1 WebView Tab Pooling

Rendering every tab as a live WebView simultaneously would destroy performance. Instead, maintain a pool of at most 3 active WebView instances at once: the current tab and one neighbour in each direction. All other tabs are unmounted and their URL/scroll state is preserved in the store. When a suspended tab becomes active, remount the WebView and restore navigation via the source.uri prop. This is the single most important performance decision in the project.

9.2 Reanimated Sheet Animation

The tab tray and menu sheet both animate along the Y axis. Use a single shared value (sheetY) per sheet, driven by a PanGestureHandler. On gesture end, use withSpring to snap to either 0 (open) or the sheet height (closed) based on velocity and position thresholds. The spring config should be: damping 20, stiffness 200, mass 0.8.

9.3 Favicon Extraction

The WebView onNavigationStateChange callback provides the title but not the favicon. Inject a small JavaScript snippet into each WebView via injectedJavaScript that reads

document.querySelector('link[rel~="icon"]')?.href and posts it back to React Native via postMessage. Listen for this in the onMessage handler and update the tab store.

9.4 Tracker Blocking

Implement a basic tracker blocker using the WebView's onShouldStartLoadWithRequest callback. Maintain a static blocklist (top 20 tracker domains as a Set<string>) and return false if the request URL's hostname is in the blocklist. Note: this only blocks navigations, not subresource loads. For subresource blocking a native module or service worker would be needed — note this limitation in a TODO comment.

9.5 Deep Linking

Configure expo-linking with the scheme zen-mobile. The /fxa-oauth path handles the Firefox Accounts OAuth redirect. Parse the authorization code from the URL query params in the Linking event listener and pass it to fxaService.handleOAuthCallback(code).

---

10. Quality Bars & Non-Negotiables

The following must all be true before the POC is considered complete. These are explicit checkboxes for the implementing agent.

- The app compiles without warnings on both iOS and Android simulators.
- TypeScript strict mode is enabled and there are zero type errors.
- The tab tray opens and closes via swipe gesture with no jank (test on a mid-range Android device or simulator).
- Switching workspaces updates the URL bar content within a single frame.
- Opening a URL (typed or from history) loads the page in the WebView.
- The Firefox Account sign-in flow opens without crashing (even if actual auth fails due to OAuth credentials not being configured).
- Bookmarking a tab persists across an app restart.
- The dark/light theme toggle updates all shell components without a full re-render.
- No component file exceeds 200 lines — split into smaller components if necessary.

---

11. Out of Scope for This POC

The following items are explicitly deferred to a future iteration. Do not implement them, and do not let their absence block the POC from being shipped.

- GeckoView integration on Android (use WebView for now).
- Full client-side sync encryption (stub the encrypt/decrypt functions with identity functions for the POC; add a TODO comment).

- Push notifications for sync conflicts.
- Extensions / userscript support.
- Reader mode.
- Picture-in-picture video.
- Desktop browser companion app pairing.

---

12. How to Use This Prompt

Paste the entire content of this document into your AI coding agent as the initial system or user message. If using Claude Projects or a similar persistent context feature, attach this document to the project context so the agent can reference it throughout the build.

The recommended build order is: types.ts → theme/index.ts → browserStore.ts → BrowserScreen.tsx skeleton → WebViewWrapper.tsx → URL bar → tab tray → gesture layer → menu sheet → FxA service stub → settings screen. Do not skip ahead to sync before the gesture layer is working — sync depends on a functioning store, which depends on working UI to validate it.

If the agent produces a file that exceeds 200 lines, instruct it to refactor immediately before continuing. Keeping files small is the best way to avoid the agent losing context of earlier decisions.